

Tensorflow II

Transfer Learning and Pretrained CNNs

Fatemeh Dehrouyeh



- 1 Transfer Learning and Pretrained CNNs
 - Motivation
 - Pretrained Models in TensorFlow
 - Preprocessing for Pretrained Models
 - Fine-Tuning
 - Evaluation and Inference
 - Choosing a Pretrained Model

Current Section

1 Transfer Learning and Pretrained CNNs

- Motivation
- Pretrained Models in TensorFlow
- Preprocessing for Pretrained Models
- Fine-Tuning
- Evaluation and Inference
- Choosing a Pretrained Model

Current Subsection

1 Transfer Learning and Pretrained CNNs

- Motivation
- Pretrained Models in TensorFlow
- Preprocessing for Pretrained Models
- Fine-Tuning
- Evaluation and Inference
- Choosing a Pretrained Model

Why Transfer Learning?

Training a CNN from scratch can require:

- a large labeled dataset,
- long training time,
- strong computational resources,
- careful model design and tuning.

Transfer learning allows us to reuse knowledge from a model that was already trained on a large dataset.

Key Idea

Instead of learning all visual features from zero, we start from a pretrained model.

What Is a Pretrained Model?

A **pretrained model** is a neural network that has already been trained on a large dataset. For image tasks, many pretrained CNNs are trained on large image datasets such as ImageNet.

These models have already learned useful visual patterns such as:

- edges,
- textures,
- shapes,
- object parts,
- high-level visual features.

Transfer Learning

We reuse these learned features for a new task.

Transfer Learning Workflow

A typical transfer-learning workflow is:

Load Pretrained Base → Remove Original Classifier → Add New Classifier → Train on New Dataset

Important

The early layers usually learn general visual patterns, while the final layers are more task-specific.

Feature Extractor vs. Classifier

A pretrained CNN can be divided into two parts:

- **Feature extractor:** convolutional base that extracts visual features.
- **Classifier:** final dense layers that make predictions.

For transfer learning, we often keep the feature extractor and replace the classifier.

Image $\rightarrow$ Pretrained Feature Extractor $\rightarrow$ New Classifier $\rightarrow$ Prediction

Current Subsection

1 Transfer Learning and Pretrained CNNs

- Motivation
- **Pretrained Models in TensorFlow**
- Preprocessing for Pretrained Models
- Fine-Tuning
- Evaluation and Inference
- Choosing a Pretrained Model

Pretrained Models in TensorFlow

TensorFlow provides many pretrained vision models through:

```
tf.keras.applications
```

Examples include:

- MobileNetV2
- ResNet50
- VGG16
- EfficientNet
- InceptionV3

In This Chapter

We will start with MobileNetV2, because it is lightweight and commonly used.

Code

MobileNetV2 is trained on ImageNet for 1000-class image classification.

```
▶ import tensorflow as tf

base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights="imagenet"
)

base_model.summary()
```

`weights="imagenet"` means TensorFlow loads MobileNetV2 with **pretrained weights learned from the ImageNet dataset**.

Instead of starting with random weights, the model already contains filters that learned useful visual features such as edges, textures, shapes, and object patterns from ImageNet.

What Does `include_top=False` Mean?

The top of a pretrained model is its original classifier.

For ImageNet models, the original classifier predicts ImageNet classes.

When we write:

```
include_top=False
```

we remove the original classifier and keep only the convolutional feature extractor.

Why?

Our new dataset may have a different number of classes, so we add our own classifier.

Code

```
▶ import tensorflow as tf

base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights="imagenet"
)

print("Output shape of feature extractor:", base_model.output_shape)
```

Freezing the Pretrained Base

At the beginning of transfer learning, we usually freeze the pretrained base.

Freezing means the pretrained weights are not updated during training.

```
base_model.trainable = False
```

Why Freeze?

We keep the useful features learned from the large dataset and train only the new classifier.

Code

```
▶ base_model.trainable = False

print("Trainable:", base_model.trainable)
print("Number of trainable weights:", len(base_model.trainable_weights))
print("Number of non-trainable weights:", len(base_model.non_trainable_weights))

... Trainable: False
   Number of trainable weights: 0
   Number of non-trainable weights: 260
```

Adding a New Classifier

After the pretrained base, we add a small classifier for our new task.

A common structure is:

Pretrained Base $\rightarrow$ Global Average Pooling $\rightarrow$ Dropout $\rightarrow$ Dense Output

Output Layer

The final dense layer should match the number of classes in the new dataset.

Code

```

num_classes = 10

inputs = tf.keras.Input(shape=(224, 224, 3))

x = base_model(inputs, training=False)
x = tf.keras.layers.GlobalAveragePooling2D()(x)
x = tf.keras.layers.Dropout(0.3)(x)

outputs = tf.keras.layers.Dense(
    num_classes,
    activation="softmax"
)(x)

model = tf.keras.Model(inputs, outputs)

model.summary()

```

... Model: "functional"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 10)	12,810

Total params: 2,270,794 (8.66 MB)

Trainable params: 12,810 (50.04 KB)

Non-trainable params: 2,257,984 (8.61 MB)

Current Subsection

1 Transfer Learning and Pretrained CNNs

- Motivation
- Pretrained Models in TensorFlow
- **Preprocessing for Pretrained Models**
- Fine-Tuning
- Evaluation and Inference
- Choosing a Pretrained Model

Preprocessing for MobileNetV2

Pretrained models usually expect images to be preprocessed in a specific way.

For MobileNetV2, TensorFlow provides:

```
tf.keras.applications.mobilenet_v2.preprocess_input
```

This transforms pixel values into the format expected by the pretrained model.

Important

Different pretrained models may require different preprocessing functions.

Image Example

```
▶ import tensorflow as tf
import urllib.request

url = "https://images.unsplash.com/photo-1591769225440-811ad7d6eab3"
urllib.request.urlretrieve(url, "dog.jpg")

img_tf = tf.keras.utils.load_img("dog.jpg")
img_tf = tf.keras.utils.img_to_array(img_tf)
```

Image

```
import matplotlib.pyplot as plt

plt.imshow(img_tf.astype("uint8"))
plt.axis("off")
plt.show()
```

...



Code

```
▶ preprocess_input = tf.keras.applications.mobilenet_v2.preprocess_input

img = tf.image.resize(img_tf, (224, 224))
img = tf.cast(img, tf.float32)

img_preprocessed = preprocess_input(img)

print("Min:", tf.reduce_min(img_preprocessed).numpy())
print("Max:", tf.reduce_max(img_preprocessed).numpy())
```

```
... Min: -1.0
    Max: 1.0
```

Building the Transfer Learning Model

The full transfer-learning model includes:

Input $\rightarrow$ Preprocessing $\rightarrow$ Pretrained Base $\rightarrow$ New Classifier

The pretrained base extracts features, and the new classifier learns the target task.

Code

```

▶ inputs = tf.keras.Input(shape=(224, 224, 3))

x = preprocess_input(inputs)
x = base_model(x, training=False)
x = tf.keras.layers.GlobalAveragePooling2D()(x)
x = tf.keras.layers.Dropout(0.3)(x)

outputs = tf.keras.layers.Dense(
    num_classes,
    activation="softmax"
)(x)

model = tf.keras.Model(inputs, outputs)

model.summary()

```

... Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 224, 224, 3)	0
true_divide (TrueDivide)	(None, 224, 224, 3)	0
subtract (Subtract)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 1280)	0
dropout_1 (Dropout)	(None, 1280)	0
dense_1 (Dense)	(None, 10)	12,810

Total params: 2,270,794 (8.66 MB)

Trainable params: 12,810 (50.04 KB)

Non-trainable params: 2,257,984 (8.61 MB)

Why CNNs Can Accept Different Image Sizes

A major advantage of convolutional layers is that their learned weights do not depend on the full image width and height.

A convolution filter may have shape:

$$3 \times 3 \times 3$$

The same filter can slide across images of different sizes:

$$224 \times 224 \times 3$$

$$160 \times 160 \times 3$$

$$128 \times 128 \times 3$$

Key Idea

The filter weights depend on:

$$\text{kernel size} \times \text{input channels}$$

They do not depend on the total image width and height.

Therefore, pretrained convolution filters can often be reused with different image resolutions.

However, when using pretrained models, we usually prefer to resize images to the input size used during pretraining, because the learned features were optimized for that scale and input configuration.

Compiling the Transfer Learning Model

When the pretrained base is frozen, only the new classifier is trained.

A common setup is:

Optimizer = Adam

Loss = Sparse Categorical Crossentropy

Metric = Accuracy

Training Goal

Learn the new classifier while keeping the pretrained features fixed.

Code

```
▶ model.compile(  
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)
```

Training Only the New Classifier

During the first transfer-learning stage:

- the pretrained base is frozen,
- only the new classifier is updated,
- training is usually faster,
- fewer parameters need to be learned.

Stage 1

Train the classifier on top of the frozen pretrained feature extractor.

Dataset

```
import pathlib
import tensorflow as tf

dataset_url = "https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz"

download_path = tf.keras.utils.get_file(
    "flower_photos",
    origin=dataset_url,
    untar=True
)

data_dir = pathlib.Path(download_path)

# Move down one level if necessary
if (data_dir / "flower_photos").exists():
    data_dir = data_dir / "flower_photos"

print("Using folder:", data_dir)
print("Folders:", [x.name for x in data_dir.iterdir() if x.is_dir()])
```

... Using folder: /root/.keras/datasets/flower_photos/flower_photos
Folders: ['sunflowers', 'roses', 'tulips', 'dandelion', 'daisy']

Dataset

```
import tensorflow as tf

train_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.30,
    subset="training",
    seed=123,
    image_size=(224, 224),
    batch_size=32
)

temp_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.30,
    subset="validation",
    seed=123,
    image_size=(224, 224),
    batch_size=32
)

# Split the 30% temporary set into 15% validation and 15% test
temp_batches = tf.data.experimental.cardinality(temp_ds)

val_ds = temp_ds.take(temp_batches // 2)
test_ds = temp_ds.skip(temp_batches // 2)

print("Classes:", train_ds.class_names)

... Found 3670 files belonging to 5 classes.
Using 2569 files for training.
Found 3670 files belonging to 5 classes.
Using 1101 files for validation.
Classes: ['daisy', 'dandelion', 'roses', 'sunflowers', 'tulips']
```

Code— Sample

```
▶ import matplotlib.pyplot as plt

class_names = train_ds.class_names

plt.figure(figsize=(10, 10))

for images, labels in train_ds.take(1):
    for i in range(9):
        ax = plt.subplot(3, 3, i + 1)

        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])

        plt.axis("off")

plt.show()
```

Code—Sample



Code

```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10
)
```

```
Epoch 1/10
81/81 ————— 138s 2s/step - accuracy: 0.6357 - loss: 1.0130 - val_accuracy: 0.8290 - val_loss: 0.4956
Epoch 2/10
81/81 ————— 118s 1s/step - accuracy: 0.8346 - loss: 0.4706 - val_accuracy: 0.8585 - val_loss: 0.3953
Epoch 3/10
81/81 ————— 118s 1s/step - accuracy: 0.8622 - loss: 0.3906 - val_accuracy: 0.8713 - val_loss: 0.3711
Epoch 4/10
81/81 ————— 142s 1s/step - accuracy: 0.8793 - loss: 0.3287 - val_accuracy: 0.8732 - val_loss: 0.3599
Epoch 5/10
81/81 ————— 138s 2s/step - accuracy: 0.9027 - loss: 0.2931 - val_accuracy: 0.8695 - val_loss: 0.3524
Epoch 6/10
81/81 ————— 124s 1s/step - accuracy: 0.9039 - loss: 0.2713 - val_accuracy: 0.8768 - val_loss: 0.3650
Epoch 7/10
81/81 ————— 141s 1s/step - accuracy: 0.9163 - loss: 0.2368 - val_accuracy: 0.8860 - val_loss: 0.3314
Epoch 8/10
81/81 ————— 142s 1s/step - accuracy: 0.9330 - loss: 0.2106 - val_accuracy: 0.8805 - val_loss: 0.3572
Epoch 9/10
81/81 ————— 140s 1s/step - accuracy: 0.9295 - loss: 0.2068 - val_accuracy: 0.8750 - val_loss: 0.3744
Epoch 10/10
81/81 ————— 152s 2s/step - accuracy: 0.9404 - loss: 0.1870 - val_accuracy: 0.8695 - val_loss: 0.3495
```

Why Start with a Frozen Base?

Freezing the base is useful because the pretrained model already contains general visual knowledge.

If we update all layers immediately, we may damage useful pretrained features, especially when the new dataset is small.

Good Practice

Start by training only the new classifier, then optionally fine-tune some deeper layers later.

Current Subsection

1 Transfer Learning and Pretrained CNNs

- Motivation
- Pretrained Models in TensorFlow
- Preprocessing for Pretrained Models
- **Fine-Tuning**
- Evaluation and Inference
- Choosing a Pretrained Model

What Is Fine-Tuning?

Fine-tuning means unfreezing part of the pretrained model and training it on the new dataset.

This allows the model to slightly adjust pretrained features for the new task.

Two-Stage Strategy

Stage 1: Train new classifier

Stage 2: Fine-tune some pretrained layers

Code

```
▶ base_model.trainable = True  
  
print("Base model trainable:", base_model.trainable)
```

Fine-Tuning Only Deeper Layers

We usually do not fine-tune all layers immediately.

Early layers often learn general features such as edges and textures.
Deeper layers learn more task-specific features.

Good Practice

Keep early layers frozen and fine-tune only later layers.

Code

```
▶ print(len(base_model.layers))
```

```
... 154
```

`fine_tune_at = 100`: Freeze layers 0 through 99, and allow layers 100 onward to train.

```
▶ fine_tune_at = 100

for layer in base_model.layers[:fine_tune_at]:
    layer.trainable = False

for layer in base_model.layers[fine_tune_at:]:
    layer.trainable = True

print("Trainable layers:")
print(sum(layer.trainable for layer in base_model.layers))
```

```
... Trainable layers:
54
```

Use a Smaller Learning Rate

During fine-tuning, we use a smaller learning rate.

Why?

- pretrained weights are already useful,
- large updates may damage learned features,
- we only want small task-specific adjustments.

Typical Idea

Fine-tuning should be gentle.

Code

```
▶ model.compile(  
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)
```

Fine-Tuning the Model

After unfreezing selected layers, continue training.

This stage adjusts the pretrained representation to the new dataset.

Important

Fine-tuning should usually be monitored carefully to avoid overfitting.

Code

```

▶ fine_tune_history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10
)

```

```

... Epoch 1/10
81/81 ————— 199s 2s/step - accuracy: 0.8034 - loss: 0.5213 - val_accuracy: 0.8952 - val_loss: 0.3357
Epoch 2/10
81/81 ————— 205s 2s/step - accuracy: 0.8840 - loss: 0.3429 - val_accuracy: 0.8897 - val_loss: 0.3271
Epoch 3/10
81/81 ————— 209s 2s/step - accuracy: 0.9140 - loss: 0.2720 - val_accuracy: 0.8897 - val_loss: 0.3190
Epoch 4/10
81/81 ————— 188s 2s/step - accuracy: 0.9179 - loss: 0.2300 - val_accuracy: 0.9007 - val_loss: 0.3147
Epoch 5/10
81/81 ————— 185s 2s/step - accuracy: 0.9299 - loss: 0.2035 - val_accuracy: 0.8989 - val_loss: 0.2860
Epoch 6/10
81/81 ————— 209s 2s/step - accuracy: 0.9432 - loss: 0.1685 - val_accuracy: 0.9007 - val_loss: 0.2931
Epoch 7/10
81/81 ————— 186s 2s/step - accuracy: 0.9506 - loss: 0.1529 - val_accuracy: 0.8971 - val_loss: 0.2872
Epoch 8/10
81/81 ————— 202s 2s/step - accuracy: 0.9615 - loss: 0.1245 - val_accuracy: 0.9081 - val_loss: 0.2716
Epoch 9/10
81/81 ————— 195s 2s/step - accuracy: 0.9692 - loss: 0.1096 - val_accuracy: 0.8989 - val_loss: 0.2669
Epoch 10/10
81/81 ————— 187s 2s/step - accuracy: 0.9669 - loss: 0.1063 - val_accuracy: 0.9081 - val_loss: 0.2532

```

Transfer Learning Summary

Transfer learning usually follows this workflow:

Load pretrained model → Freeze base → Add classifier → Train classifier → Fine-tune

Key Idea

Pretrained CNNs provide strong visual features that can be reused for new tasks.

Current Subsection

1 Transfer Learning and Pretrained CNNs

- Motivation
- Pretrained Models in TensorFlow
- Preprocessing for Pretrained Models
- Fine-Tuning
- **Evaluation and Inference**
- Choosing a Pretrained Model

Evaluating the Transfer Learning Model

After training and fine-tuning, evaluate the model on unseen test data.

The test set should be used only for final evaluation.

Goal

Estimate how well the model generalizes to new images.

Code

```
test_loss, test_acc = model.evaluate(test_ds)

print("Test loss:", test_loss)
print("Test accuracy:", test_acc)
```

18/18  **23s** 1s/step - accuracy: 0.9372 - loss: 0.2150
Test loss: 0.21501177549362183
Test accuracy: 0.9371633529663086

Making Predictions with a Pretrained Model

For inference, the image must go through the same preprocessing steps used during training.

A prediction pipeline is:

Load Image → Resize → Preprocess → Model → Prediction

Code

Search the Internet for an image of any of the following flowers and see whether the model can predict it correctly: ['sunflowers', 'roses', 'tulips', 'dandelion', 'daisy']

```
import tensorflow as tf
import matplotlib.pyplot as plt

image_path = "sunflower.png"

raw = tf.io.read_file(image_path)

img = tf.image.decode_image(
    raw,
    channels=3,
    expand_animations=False
)

img = tf.image.resize(img, (224, 224))

# Show the image
plt.imshow(img.numpy().astype("uint8"))
plt.axis("off")
plt.show()

# Add batch dimension
img_batch = tf.expand_dims(img, axis=0)

# Predict
probs = model.predict(img_batch)

pred_class = tf.argmax(probs[0]).numpy()

print("Predicted class index:", pred_class)
print("Predicted class:", train_ds.class_names[pred_class])
print("Confidence:", probs[0][pred_class])
```

Code—Result



1/1 ————— 0s 107ms/step

Predicted class index: 3

Predicted class: sunflowers

Confidence: 0.99724716

Saving the Transfer Learning Model

After training, save the complete model.

This includes:

- preprocessing layers,
- pretrained base,
- new classifier,
- learned weights.

Keras Format

```
model.save("transfer_model.keras")
```

Code



```
model.save("transfer_learning_model.keras")
```

Loading the Saved Model

A saved model can be loaded later for evaluation or prediction.

This makes it possible to reuse the trained model without training again.

Code

```

▶ loaded_model = tf.keras.models.load_model("transfer_learning_model.keras")

loaded_model.summary()

```

... Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_3 (InputLayer)	(None, 224, 224, 3)	0
true_divide (TrueDivide)	(None, 224, 224, 3)	0
subtract (Subtract)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 1280)	0
dropout_1 (Dropout)	(None, 1280)	0
dense_1 (Dense)	(None, 10)	12,810

Total params: 6,019,296 (22.96 MB)

Trainable params: 1,874,250 (7.15 MB)

Non-trainable params: 396,544 (1.51 MB)

Optimizer params: 3,748,502 (14.30 MB)

Code

```
test_loss, test_acc = loaded_model.evaluate(test_ds)

print("Loaded model test accuracy:", test_acc)
```

18/18 ————— **24s** 1s/step - accuracy: 0.9318 - loss: 0.2347
Loaded model test accuracy: 0.9317773580551147

What Can Go Wrong in Transfer Learning?

Common transfer-learning problems include:

- using the wrong preprocessing function,
- forgetting to resize images,
- using the wrong number of output classes,
- fine-tuning with a learning rate that is too large,
- applying augmentation to validation or test data,
- overfitting a small dataset.

Good Habit

Always check image shape, value range, preprocessing, and output layer size.

Current Subsection

1 Transfer Learning and Pretrained CNNs

- Motivation
- Pretrained Models in TensorFlow
- Preprocessing for Pretrained Models
- Fine-Tuning
- Evaluation and Inference
- **Choosing a Pretrained Model**

Which Pretrained Model Should We Use?

TensorFlow provides many pretrained CNNs.

The best choice depends on the application:

- **MobileNetV2**: lightweight and efficient
- **ResNet50**: strong general-purpose baseline
- **VGG16**: simple and historically important
- **EfficientNet**: strong accuracy–efficiency trade-off
- **InceptionV3**: multi-scale feature extraction

Practical Rule

Use smaller models for limited resources and larger models when accuracy is more important.

Code

```
models = {  
    "MobileNetV2": tf.keras.applications.MobileNetV2,  
    "ResNet50": tf.keras.applications.ResNet50,  
    "VGG16": tf.keras.applications.VGG16,  
    "EfficientNetB0": tf.keras.applications.EfficientNetB0,  
}  
  
for name, model_fn in models.items():  
    base = model_fn(  
        input_shape=(224, 224, 3),  
        include_top=False,  
        weights="imagenet"  
    )  
    print(name, "parameters:", base.count_params())
```

```
... MobileNetV2 parameters: 2257984  
    ResNet50 parameters: 23587712  
    VGG16 parameters: 14714688  
    EfficientNetB0 parameters: 4049571
```

Model Size vs. Accuracy

Pretrained models differ in complexity.

Larger models may learn richer features, but they often require:

- more memory,
- more computation,
- longer inference time,
- more energy on edge devices.

Key Trade-off

Computer vision is often a balance between **accuracy**, **speed**, and **resource usage**.

End of Transfer Learning Chapter

In this chapter, we learned how to:

- load pretrained CNNs,
- remove the original classifier,
- freeze pretrained layers,
- add a new classifier,
- train the classifier,
- fine-tune deeper layers,
- evaluate and save the final model.

Next Chapter

We now move from CNN-based vision models to modern vision architectures: **Vision Transformers**.

Congratulations!

Now you know about transfer learning!

